

1
0
0
1
0
1
0
1
0
1
0
1
0
1
0
1
0
0

스마트팜 통합 솔루션 포트폴리오

지원자 : 이의락



1
0
1
0
0
1
0
1
0
1
0
0
1
0
1
0

목차

01

작품 소개

02

아키텍처

03

진행 과정

04

샘플코드

1
0
1
0
1
0
1
1
1
1
0
0
0
1
1
1
0

1
1
0
0
1
0
1
0
1
0
1
0
1
1
1
1
0



1
0
1
0
1
0
1
1
1
1
0
0
0
1
1
1
0



01

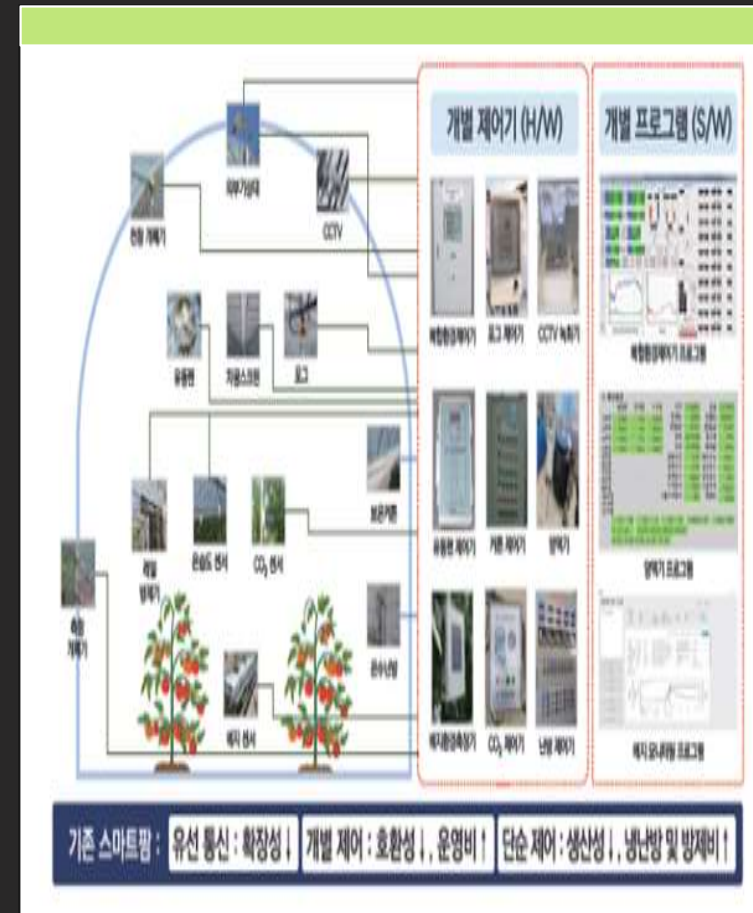
작품 소개

1
1
0
0
1
0
1
0
1
0
1
1
1
1
0

프로젝트 배경

기존 스마트팜 문제점 분석

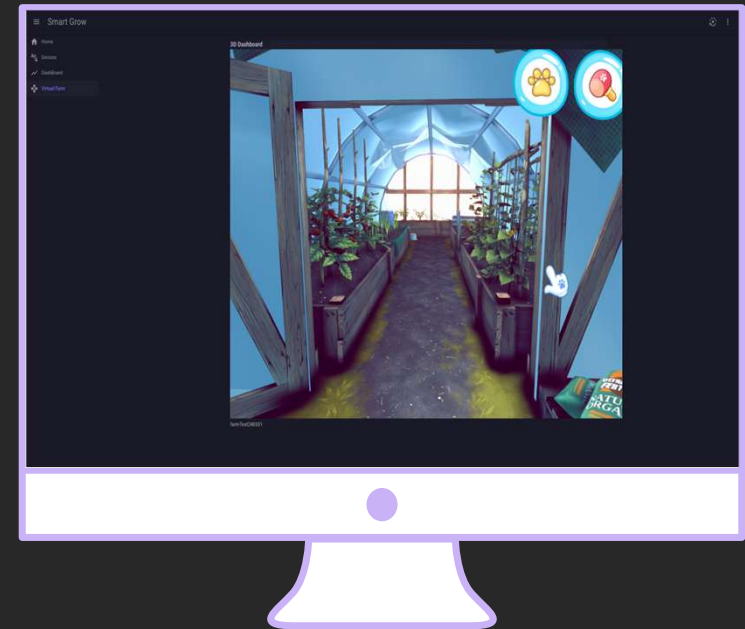
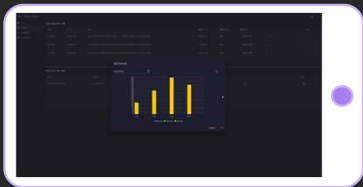
- ✓ 확장성 : 새로운 스마트팜 장비를 사용하는데 기존 장비들과 독립적으로 개발되어 호환하기 어려움.
- ✓ 제어성 : 새로운 스마트팜 장비를 제어하는데 독립된 소프트웨어를 추가해야 해서 새로 배우고 숙달해야 하므로 사용하기 불편함.
- ✓ 사용성 : 옛날 그래픽의 2D UI로 표현되어 있어서 실물과 대조하기 어려우며 전문 용어 또는 제품명으로 숙지하여 사용하여야 하기에 상당히 불편함.



작품 소개



1
0
0
1
0
1
0
1
0
0
1
0
1
0
1
0



1
0
1
0
0
1
0
0
1
0
1
1
0
1
1
1

스마트팜 통합 솔루션

모든 스마트팜 장비를 하나의 프로그램에서 전장 제어할 수 있는 디지털 트윈 소프트웨어

기대 효과



업무 간소화

스마트팜 장비별로 연동되는
여러 프로그램을 유지 관리했던
업무들이 하나의 프로그램으로
업무량이 간소화되어 시간 및
업무 효율 증진함.



신기술 확장

다양한 장치와 프로그램을
원활하게 통신할 수 있도록
표준화된 인터페이스 및 새로운
기술들도 확장 가능한 기회
제공함.



기술 장벽 완화

디지털 트윈 기술로 게임처럼
사용자 친화적이고 매력적으로
설계되어 복잡한 스마트팜
시스템을 더 쉽게 사용하고
배울 수 있는 편의성 제공함.

1
0
0
1
0
1
0
1
0
0
1
0
1
0
1
0

1
0
1
0
0
1
0
0
1
0
1
0
1
1
1
1



1
0
1
0
1
0
1
1
1
1
0
0
0
1
1
1
0



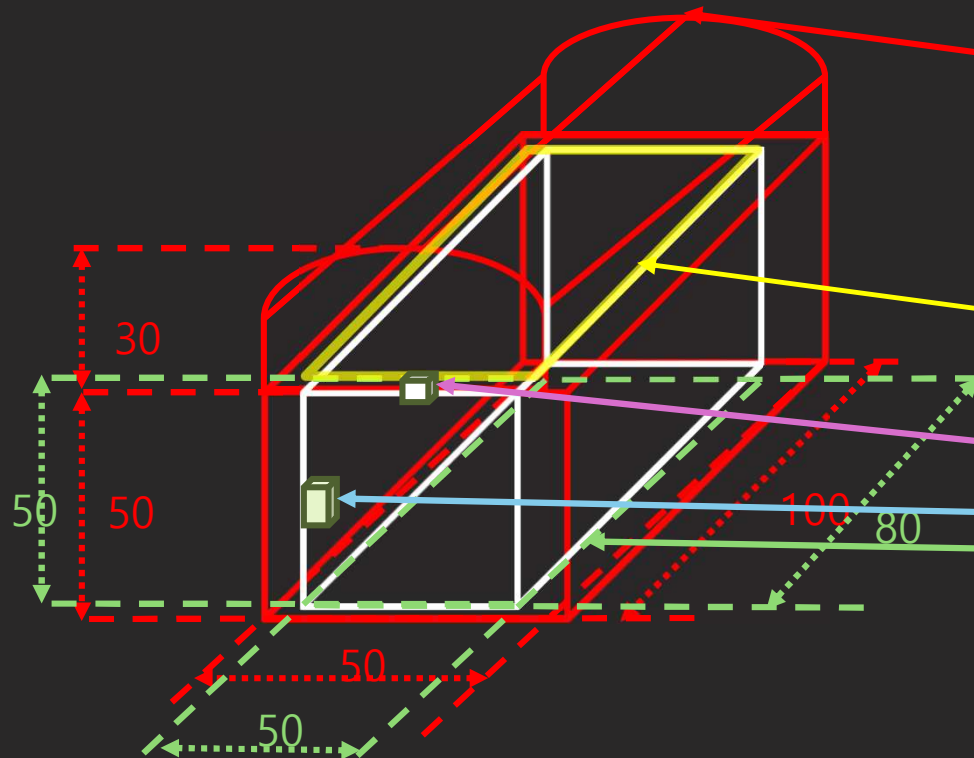
02

아키텍처

1
1
0
0
1
0
1
0
1
0
1
0
1
1
1
0

하드웨어

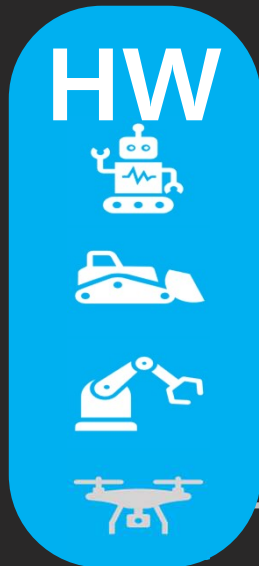
1
0
0
1
0
1
0
1
0
0
1
0
1
0
1
0
1
0



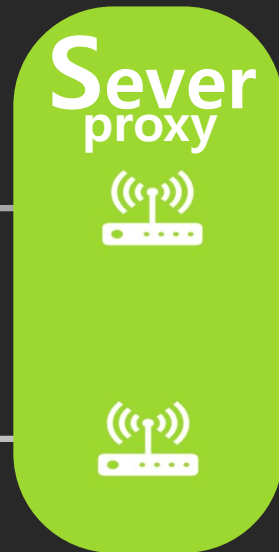
1
0
1
0
0
1
0
0
1
0
1
0
1
1
1
1
1

소프트웨어

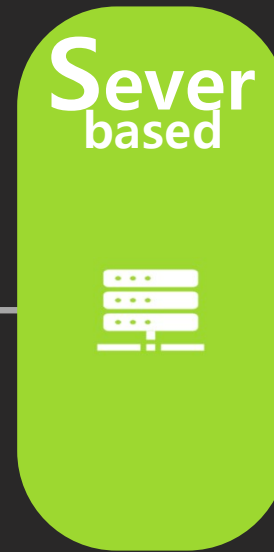
1
0
0
1
0
1
0
1
0
0
1
0
1
0
1
0



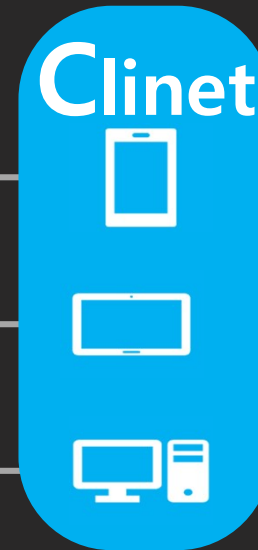
- 아두이노
- 각 센서 제어
- 와이파이 통신



- 라즈베리 파이
- HW 데이터 입/출력 제어
- 와이파이/ 유선랜 통신



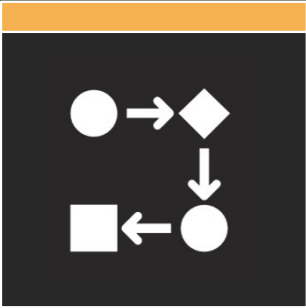
- 리눅스 컴퓨터
- 데이터 저장 및 분석
- 유선랜 통신



- 여러 OS 장비
- WebAssembly 사용
- 유선랜 통신

1
0
1
0
0
1
0
0
1
0
1
0
1
1
0
1
1

1
0
1
0
1
0
1
1
1
1
0
0
0
1
1
1
0

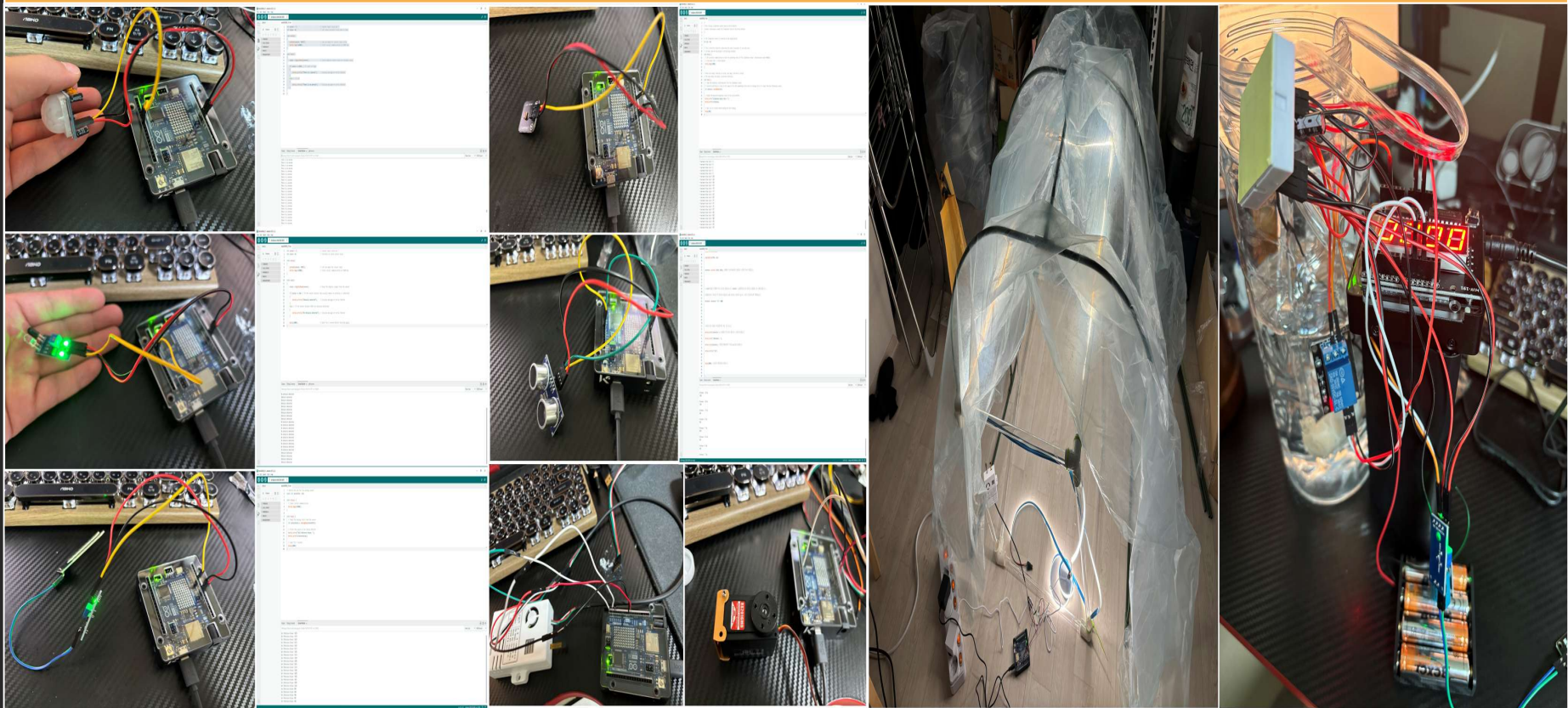


03

진행과정

1
1
0
0
1
0
1
0
1
0
1
1
1
0

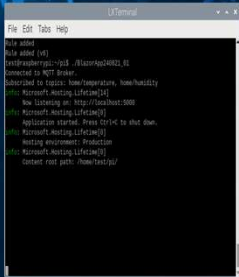
하드웨어 제작



1
0
0
1
0
1
0
1
0
0
1
0
1
0
1
0
1
0

1
0
1
0
0
1
0
0
1
0
1
1
0
1
1
1
1

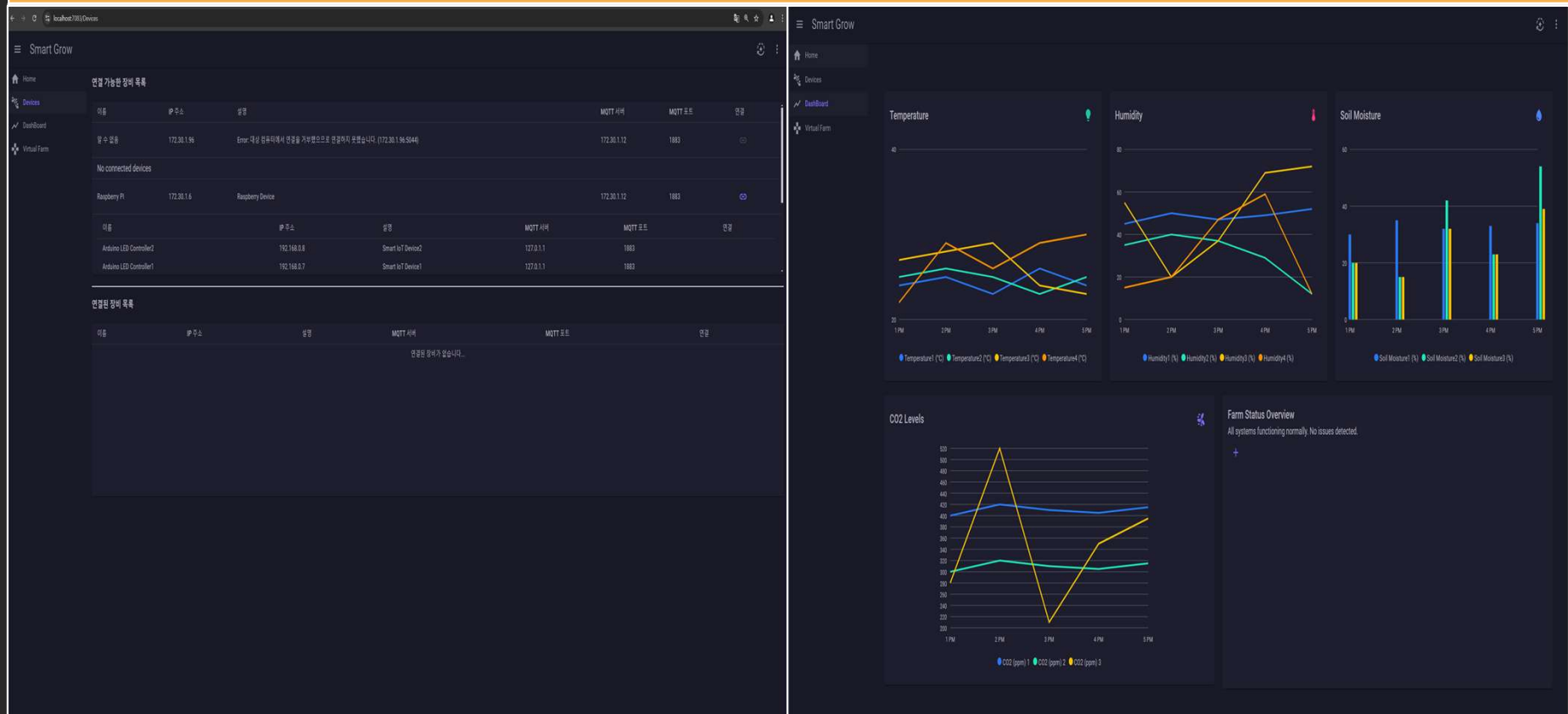
1
0
0
1
0
1
0
1
0
0
1
0
1
0
1
0



1
0
0
1
0
1
0
1
0
0
1
0
1
0
1
0

10010010010111

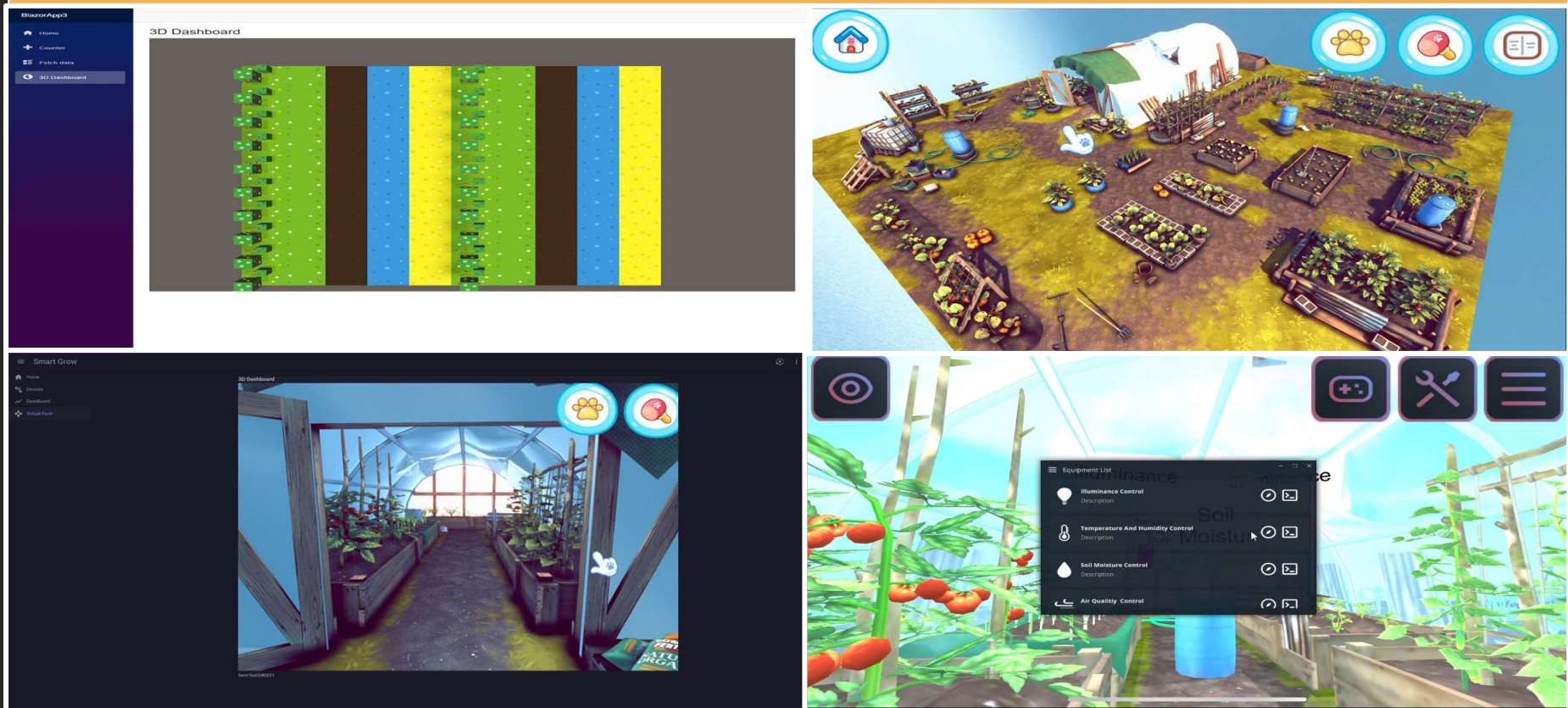
Blazor UI 개선



1
0
0
1
0
1
0
1
0
1
0
1
0
1
0
1
0
1
0

1
0
1
0
0
1
0
0
1
0
0
1
0
1
1
1
1

WebGL 적용

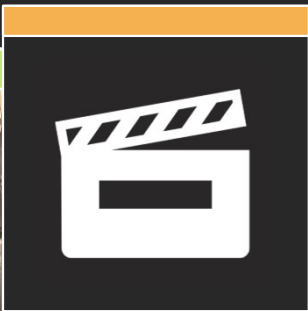


1
0
0
1
0
1
0
1
0
0
1
0
1
0
1
0
1
0

1
0
1
0
0
1
0
0
1
0
0
1
0
1
0
1
1
1



1
0
1
0
1
0
1
1
1
1
0
0
0
1
1
1
0



04

샘플 코드

1
1
0
0
1
0
1
0
1
0
1
0
1
1
1
0

아두이노

LEDSensor 예제 코드

- ✓ LED와 조도센서와 MQTT 통신 설정을 미리 라이브러리로 만들어서 사용하여 코드 재사용성, 유지보수 및 확장성을 높였습니다.
- ✓ 처음 연결은 HTTP 통신으로 사용자 원격으로 연결을 할 수 있게 하였으며, 라즈베리 파이로부터 MQTT 통신할 데이터를 주고 받을 수 있습니다.
- ✓ HTTP통신 대신 MQTT 통신을 통해 저전력 소비, 발행 및 구독 유연성, 네트워크 대역폭 절약, 실시간 효율을 높였습니다.

```

1  #include "MQTTManager_unoR4wifi.h"
2  #include "LightSensor_unoR4wifi.h"
3  #include "LED_unoR4wifi.h"
4
5  // MQTTManager 라이브러리 인스턴스
6  MQTTManager* mqttManager;
7
8  // 센서 변수 선언
9  LightSensor* sensorEast;
10 LightSensor* sensorWest;
11 LightSensor* sensorSouth;
12 LightSensor* sensorNorth;
13
14 // LED 선언
15 LED* led;
16
17 void setup() {
18     Serial.begin(115200);
19
20     const char* ssid = "한이파에 기기";
21     const char* password = "한이파에 비밀번호";
22     const char* deviceName = "Uno R4 wifi LED Controller1";
23     const char* deviceDescription = "Smart IoT Device with ESP32";
24
25     // MQTTManager 라이브러리 인스턴스 초기화
26     mqttManager = MQTTManager::getInstance(ssid, password, deviceName, deviceDescription);
27     mqttManager->begin(); // Wi-Fi 및 MQTT 연결 시도
28
29     // 각 센서 변수 객체 생성
30     sensorEast = new LightSensor(A0, "sensor/east", mqttManager);
31     sensorWest = new LightSensor(A1, "sensor/west", mqttManager);
32     sensorSouth = new LightSensor(A2, "sensor/south", mqttManager);
33     sensorNorth = new LightSensor(A3, "sensor/north", mqttManager);
34
35     // LED 객체 생성 (ESP32에서는 GPIO 2번 핀 사용)
36     led = new LED(2);
37
38     // MQTT 토픽 등록 (필요할 경우)
39     mqttManager->addTopic("led", "home/led");
40     mqttManager->addTopic("east", "sensor/east");
41     mqttManager->addTopic("west", "sensor/west");
42     mqttManager->addTopic("south", "sensor/south");
43     mqttManager->addTopic("north", "sensor/north");
44
45     // MQTT 콜백 설정 (LED 제어)
46     mqttManager->setCallback(LED::mqttCallback);
47 }
48
49 void loop() {
50     // MQTT 연결 이벤트 확인
51     mqttManager->handleClient();
52
53     // 각 센서의 값을 읽고 MQTT에 전송
54     int eastLight = sensorEast->readLightLevel();
55     if (!mqttManager->isConnected()) {
56         // Serial.println("MQTT에 연결되지 않았습니다.");
57     } else {
58         mqttManager->publishMessage("sensor/east", String(eastLight).c_str());
59     }
60     int westLight = sensorWest->readLightLevel();
61     mqttManager->publishMessage("sensor/west", String(westLight).c_str());
62
63     int southLight = sensorSouth->readLightLevel();
64     mqttManager->publishMessage("sensor/south", String(southLight).c_str());
65
66     int northLight = sensorNorth->readLightLevel();
67     mqttManager->publishMessage("sensor/north", String(northLight).c_str());
68
69     // 타이밍을 맞추기 위해 delay
70     delay(5000);
71 }

```

1010010101010101010

라즈베리 파이

DeviceController 예제 코드

- ✓ 현재 같은 망에 연결되어 있는 장비들에게 Ping을 보내서 응답여부를 보고 연결 가능한 장비를 조회한다.
- ✓ 그리고 해당 라즈베리 파이의 장비 정보를 클라이언트에게 보내서 해당 망에 연결된 아두이노에 대한 정보를 라즈베리 파이 하위 항목으로 보여준다.
- ✓ MQTT토픽 정보는 아두이노가 가지고 있으며 라즈베리 파이와 연결할 때 HTTP통신으로 가져온다.

```

22 // 라즈베리 파이 장치 검색 API
23 [HttpGet("device_info")]
24 public async Task<ActionResult> GetDeviceInfo ()
25 {
26     Console.WriteLine("연결이 시작되었다.");
27     List<Device> devices = await _deviceDiscoveryService.DiscoverDevicesAsync();
28     Dictionary<string, List<string>> totalTopics = new Dictionary<string, List<string>>();
29     Console.WriteLine("devices 개수 : " + devices.Count);
30     List<Device> arduinoDevices = new List<Device>();
31     foreach (var device in devices)
32     {
33         device.MqttServer = _deviceDiscoveryService._serverIpAddress;
34         device.MqttPort = "1883";
35
36         Console.WriteLine("device.Address : " + (device?.Address == null ? "0.0.0.0" : device.Address) +
37             " device.MqttTopics 개수 : " + (device?.MqttTopics == null ? 0 : device.MqttTopics.Count));
38
39         if (device.MqttServer == null || device.MqttPort == null || device.MqttTopics == null)
40             continue;
41         // 첫 번째 토픽을 string으로 ConnectAsync에 전달
42         await _mqttService.ConnectAsync(device.MqttServer, int.Parse(device.MqttPort), device.MqttTopics);
43         foreach (var t in device.MqttTopics.ToDictionary(pair => pair.Key, pair => pair.Value))
44         {
45             Console.WriteLine("t.Key : " + t.Key);
46             Console.WriteLine("t.Value : " + t.Value);
47
48             totalTopics.Add(device.Name + "-" + t.Key, t.Value);
49         }
50         arduinoDevices.Add(device);
51     }
52     Console.WriteLine("아두이노 연결이 종료되었다. : " + totalTopics.Count);
53
54     Device raspberry = new Device
55     {
56         Name = "Raspberry PI",
57         Address = _deviceDiscoveryService._serverIpAddress,
58         Description = "Raspberry Device",
59         MqttServer = _deviceDiscoveryService._serverIpAddress,
60         MqttPort = "1883",
61         MqttTopics = totalTopics,
62         ConnectedDevices = arduinoDevices
63     };
64     Console.WriteLine("ConnectedDevices : " + (raspberry?.ConnectedDevices?.Count == null ? "없음" : raspberry?.ConnectedDevices?.Count));
65
66     return Ok(raspberry);
67 }

```

1
0
1
0
0
1
0
1
0
1
0
1
0
1
0

Devices 예제 코드

- ✓ 연결되는 장비 정보에 null과 같은 문제가 있는 것은 “알 수 없음” 표시합니다.
- ✓ 연결되는 장비에 자식 디바이스가 있으면 똑같은 객체로 확장 표시로 추가하여 보여줍니다.
- ✓ 이때 자식의 객체에는 토픽을 Tree구조로 모아서 보여주며 해당 토픽 및 장비들의 정보를 보고 연결하여서 사용하고 싶으면 ‘연결’을버튼을 클릭하여 연결합니다.

[illegible]

1
0
1
0
0
1
0
1
0
1
0
0
1
0
1
0

유니티 WebGL

unityInstanceLoader 예제 코드

- ✓ Unity WebGL에서 버튼 클릭 이벤트가 발생하였을 때, Blazor WebAssembly와 상호작용하는 것을 구현한 JavaScript Interop코드입니다.
- ✓ DotNet.invokeMethodAsync 함수를 이용하여 Blazor WebAssembly에서 C# 메서드를 호출하여 해당 Dialog를 열도록 설계하였습니다.
- ✓ MergeInto 함수로 Unity WebGL에 사용하는 라이브러리 객체에 새로운 기능을 추가하며, 여러 컨트롤 함수들을 병합하고 있습니다.

```

1 mergeInto(LibraryManager.Library, {
2   LedControl: function() {
3     console.log("LED Control button clicked in Unity WebGL.");
4     DotNet.invokeMethodAsync("MudBlazorWebApp240916.Client", "ShowIlluminanceControlDialog", "LedControl");
5   },
6   WaterControl: function() {
7     console.log("Water Control button clicked in Unity WebGL.");
8     DotNet.invokeMethodAsync("MudBlazorWebApp240916.Client", "ShowSoilMoistureDialog", "WaterControl");
9   },
10  AirControl: function() {
11    console.log("Air Control button clicked in Unity WebGL.");
12    DotNet.invokeMethodAsync("MudBlazorWebApp240916.Client", "ShowAirQualityDialog", "AirControl");
13  },
14  TemperatureAndHumidityControl: function() {
15    console.log("TemperatureAndHumidityControl button clicked in Unity WebGL.");
16    DotNet.invokeMethodAsync("MudBlazorWebApp240916.Client", "ShowTemperatureAndHumidityDialog", "TemperatureAndHumidityControl");
17  },
18  MemoList: function() {
19    console.log("Memo list button clicked in Unity WebGL.");
20    DotNet.invokeMethodAsync("MudBlazorWebApp240916.Client", "ShowMemoListDialog", "MemoList");
21  }
22 });
23

```

1
0
1
0
0
1
0
1
0
1
0
0
1
0
1
0